

JavaScript Deep-Dive Flashcards - Questions & Answers

Category: this Keyword

Q173: What is `this` in the global scope in a browser?

Context: In a browser script (non-module), what does `console.log(this);` output?

Answer: `window` — in a browser's global scope, `this` refers to the `window` object.

Q174: What does `this` resolve to inside a regular (non-arrow) function called without any context, in strict mode?

Context: Consider: `"use strict"; function whoAmI() { return this; } console.log(whoAmI());`

Answer: `undefined` — strict mode does not default `this` to the global object.

Q175: What does `this` resolve to inside the same function in non-strict (sloppy) mode?

Context: Example: `function whoAmI() { return this; } console.log(whoAmI());`

Answer: `window` (browser) / `global` (Node) — in sloppy mode, a bare function call defaults `this` to the global object.

Q176: What is `this` inside the nested `inner` function?

Context: Example: `const obj = { name: "Venky", outer() { function inner() { return this; } return inner(); } }; console.log(obj.outer());`

Answer: `undefined` in strict mode, `window` in sloppy mode — `inner()` is called as a bare function, not as a method. It loses `obj`'s context.

Q177: How can you fix the nested function in Q176 so `inner` correctly reads `this.name`?

Context: Example: `const obj = { name: "Venky", outer() { const inner = () => this.name; return inner(); } }; console.log(obj.outer());`

Answer: Both Option A (arrow function) and Option B (`self` pattern) work — arrow captures lexical `this`; `self` manually preserves it.

Q178: What does `this` refer to inside a `.then()` callback?

Context: Example: `const obj = { name: "Venky", fetch() { return Promise.resolve().then(function() { return this; }); } }; obj.fetch().then(val => console.log(val));`

Answer: `undefined` in strict mode, `window` in sloppy mode — `.then()` callbacks are called as bare functions, not methods.

Q179: You destructure a method off an object. What happens to `this`?

Context: Example: `const obj = { name: "Venky", greet() { return `Hi ${this.name}`; } }; const { greet } = obj; console.log(greet());`

Answer: `"Hi undefined"` — `greet` is now a bare function reference. Calling it without `obj` loses the context, so `this.name` is `undefined`.

Q180: What is the `this` priority order when multiple binding mechanisms compete?

Context: Rank from highest to lowest.

Answer: `new` > `.bind()` > `.call()/.apply()` > implicit (method call) > global/default.

Q181: `new` keyword vs `.bind()` — which wins?

Context: Example: `function Person(name) { this.name = name; } const BoundPerson = Person.bind({ name: "forcedName" }); const p = new BoundPerson("ActualName"); console.log(p.name);`

Answer: `"ActualName"` — `new` overrides `.bind()`. When used with `new`, `this` is always the freshly created object.

Q182: What is `this` inside a getter?

Context: Example: `const obj = { _name: "Venky", get name() { return this._name; } }; console.log(obj.name);`

Answer: `obj` — getters are called as methods on the object, so `this` refers to the object they belong to.

Q183: What is `this` inside a `setTimeout` callback in a class method?

Context: Example: `class Timer { constructor() { this.seconds = 0; } start() { setTimeout(function() { this.seconds++; }, 1000); } } new Timer().start();`

Answer: `undefined` in strict mode, `window` in sloppy mode — `setTimeout` calls its callback as a bare function.

Category: Arrow Functions

Q184: What happens if you try to access `arguments` in an arrow function?

Context: Example: `const fn = () => console.log(arguments); fn(1, 2, 3);`

Answer: Throws a `ReferenceError` — arrow functions do not have their own `arguments` object.

Q185: Can an arrow function be used as a constructor?

Context: Example: `const Person = (name) => { this.name = name; }; const p = new Person("Venky");`

Answer: No — arrow functions cannot be used with `new`. They throw a `TypeError: "Person is not a constructor"`.

Q186: What is the output of an arrow function inside an object literal?

Context: Example: `const obj = { name: "Venky", greet: () => `Hi ${this.name}` }; console.log(obj.greet());`

Answer: `"Hi undefined"` — the arrow function captures `this` from its surrounding lexical scope (the global/module scope), NOT from `obj`.

Q187: Arrow function inside a class field vs a method: what is the output?

Context: Example: `class Dog { name = "Rex"; barkArrow = () => this.name; barkMethod() { return this.name; } } const dog = new Dog(); const { barkArrow, barkMethod } = dog; console.log(barkArrow()); console.log(barkMethod());`

Answer: `barkArrow` logs `"Rex"`, `barkMethod` logs `undefined` — class field arrows bind `this` to the instance at creation. Destructuring the method loses context.

Q188: What's the gotcha with arrow functions returning an object literal?

Context: Example: `const makeUser = (name) => { name: name, age: 25 }; console.log(makeUser("Venky"));`

Answer: `undefined` — the `{ }` is interpreted as a block, not an object literal. The fix is to wrap in parentheses: `=> ({ name, age: 25 })`.

Q189: Do arrow functions have a `.prototype` property?

Context: Example: `const Fn = () => {}; console.log(Fn.prototype);`

Answer: `undefined` — arrow functions have no `.prototype` because they can't be constructors.

Q190: How do you fix an arrow function inside `.map()` to preserve `this`?

Context: Example:

```
class UserList { constructor(users) { this.users = users; } getNames() { return this.users.map(function(u) { return u + " from " + this.company; }); } } new UserList(["A","B"]).getNames();
```

Answer: `this.company` is `undefined` inside the `.map()` callback because it's a regular function. Fix: use an arrow function `(u) => u + " from " + this.company`.

Category: Binding

Q191: You `.bind()` a function twice. Which binding wins?

Context: Example:

```
function greet() { return this.name; } const bound1 = greet.bind({ name: "First" }); const bound2 = bound1.bind({ name: "Second" }); console.log(bound2());
```

Answer: `"First"` — once a function is hard-bound with `.bind()`, subsequent `.bind()` calls cannot override `this`.

Q192: `.bind()` vs `.call()` — when do you use which?

Context: What is the difference?

Answer: `.bind()` returns a new function for later invocation; `.call()` invokes immediately.

Q193: What is partial application with `.bind()`?

Context: Example:

```
function add(a, b, c) { return a + b + c; } const add5and10 = add.bind(null, 5, 10); console.log(add5and10(20));
```

Answer: `35` — `bind(null, 5, 10)` fixes `a=5`, `b=10`. Calling `add5and10(20)` fills `c=20`. Result: `5+10+20 = 35`.

Q194: `.call()` with spread vs `.apply()` — are they equivalent now?

Context: Example:

```
const args = [1, 2, 3]; fn.call(ctx, ...args); fn.apply(ctx, args);
```

Answer: Yes, they are equivalent in modern JS — ES6 spread makes `.apply()` largely unnecessary, but both achieve the same result.

Category: Call Stack

Q195: What happens when you have infinite recursion in JavaScript?

Context: Example: `function infinite() { return infinite(); } infinite();`

Answer: A `RangeError: Maximum call stack size exceeded` is thrown.

Q196: Which of these will NOT cause a stack overflow even with deep recursion?

Context: Example: `function sumA(n) { if (n === 0) return 0; return n + sumA(n - 1); } function sumB(n, acc = 0) { if (n === 0) return acc; return sumB(n - 1, acc + n); }`

Answer: Option B — in engines that support Tail Call Optimization (TCO), the tail-recursive form reuses the same stack frame.

Q197: What happens to the call stack when you `await` a promise?

Context: Example: `async function A() { console.log("before"); await somePromise; console.log("after"); }`

Answer: `A` is removed from the call stack at the `await` point — execution is suspended and the stack unwinds. When the promise resolves, `A` resumes as a microtask.

Q198: Why can't you 'yield' back to the browser mid-loop in a long synchronous loop?

Context: Example: `for (let i = 0; i < 1000000; i++) { heavyComputation(i); }`

Answer: The call stack is occupied by the loop. The event loop cannot process any other tasks (paint, input, timers) until the synchronous call stack is completely empty.

Q199: How do you fix the blocking loop in Q199 to keep the UI responsive?

Context: Example: `for (let i = 0; i < 1000000; i++) { heavyComputation(i); }`

Answer: Use a `Web Worker`, break the loop into chunks using `setTimeout(chunk, 0)`, or use `requestIdleCallback`.

Category: Heap & Memory

Q201: Where is this data stored — stack or heap?

Context: Example: `const x = 42; const obj = { name: "Venky" }; const arr = [1, 2, 3];`

Answer: `x` on stack; `obj` and `arr` on heap — primitives are stored on the stack, objects/arrays on the heap.

Q202: What does this demonstrate about reference vs value types?

Context: Example: `const a = { val: 1 }; const b = a; b.val = 999; console.log(a.val);`

Answer: `999` — `b` and `a` point to the same object on the heap. Mutating through `b` is visible through `a`.

Q203: An object has a circular reference. Are `a` and `b` garbage collected after the function returns?

Context: Example: `function createCycle() { const a = {}; const b = { ref: a }; a.ref = b; } createCycle();`

Answer: Yes — mark-and-sweep traces reachability from root objects. Once `createCycle` returns, neither `a` nor `b` is reachable.

Q204: What is a `WeakMap` and why does it matter for memory?

Context: Example: `const cache = new WeakMap(); let obj = { id: 1 }; cache.set(obj, "cached data"); obj = null;`

Answer: The cache entry is eligible for garbage collection once `obj` has no other references — `WeakMap` holds weak references to keys.

Q205: When would you use `WeakRef`?

Context: Example: `let heavy = { data: new Array(1000000) }; const ref = new WeakRef(heavy); heavy = null; console.log(ref.deref());`

Answer: May return the object or `undefined` — `WeakRef` does not prevent garbage collection.

Q206: What breaks V8's hidden class optimization?

Context: Example: `function createUser(name, age) { const user = { name }; if (age) user.age = age; return user; }`

Answer: Option B — dynamically adding/removing properties creates multiple hidden classes, slowing property access.

Category: Execution Context

Q207: How many execution contexts exist when this runs?

Context: Example: `function outer() { function inner() { console.log("hello"); } inner(); } outer();`

Answer: 3 — Global EC → `outer` EC → `inner` EC.

Q208: What is the difference between the Variable Environment and the Lexical Environment inside an execution context?

Context: What is the difference?

Answer: Variable Environment stores `var` declarations and function declarations; Lexical Environment extends it by also handling `let`, `const`, and block scoping.

Q209: What is a closure, in terms of execution context?

Context: Example: `function makeCounter() { let count = 0; return () => ++count; } const counter = makeCounter();`

Answer: The returned arrow function retains a reference to `makeCounter`'s execution context (its Lexical Environment) even after `makeCounter` has returned.

Q210: Why does this work with an IIFE?

Context: Example: `((function() { var secret = "hidden"; })); console.log(secret);`

Answer: `ReferenceError: secret is not defined` — the IIFE creates its own execution context with its own Variable Environment.

Q211: Block scope vs function scope — what is the output?

Context: Example: `{ var x = 10; let y = 20; } console.log(x); console.log(y);`

Answer: `10, ReferenceError` — `var` is function-scoped (or global if not inside a function), so it leaks out of the block. `let` is block-scoped.

Category: Scope & Hoisting

Q212: What is the output?

Context: Example: `console.log(a); var a = 5; console.log(a);`

Answer: `undefined, 5` — `var a` is hoisted to the top of the scope (declaration only).

Q213: What is the output?

Context: Example: `console.log(b); let b = 10;`

Answer: `ReferenceError: Cannot access 'b' before initialization` — `let` is hoisted but NOT initialized.

Q214: Function declaration vs function expression hoisting: what is the output?

Context: Example: `console.log(declaredFn()); console.log(expressionFn()); function declaredFn() { return "declared"; }
var expressionFn = function() { return "expression"; };`

Answer: `"declared"`, then `TypeError: expressionFn is not a function` — function declarations are fully hoisted.

Q215: Shadowing — what is the output?

Context: Example: `let x = "global"; function test() { let x = "local"; console.log(x); } test(); console.log(x);`

Answer: `"local", "global"` — the inner `let x` shadows the outer one inside `test`.

Q216: What is the output?

Context: Example: `var x = 1; function test() { console.log(x); var x = 2; console.log(x); } test();`

Answer: `undefined, 2` — `var x` inside `test` is hoisted to the top of the function.

Q217: What is the output?

Context: Example: `console.log(foo()); function foo() { return "function declaration"; } var foo = function() { return "function expression"; };`

Answer: `"function declaration"` — the function declaration is fully hoisted first.

Category: Prototypes & Inheritance

Q218: What is the difference between `__proto__` and `.prototype`?

Context: What is the difference?

Answer: `.prototype` is a property on constructor functions; `__proto__` is the actual prototype link on every object, pointing up the chain.

Q219: What is the output?

Context: Example:

```
function Animal(name) { this.name = name; } Animal.prototype.speak = function() { return this.name + " speaks"; }; const dog = new Animal("Rex"); console.log(dog.hasOwnProperty("name")); console.log(dog.hasOwnProperty("speak"));
```

Answer: `true, false` — `name` is set directly on the instance via `this.name`. `speak` lives on `Animal.prototype`.

Q220: `instanceof` gotcha — what is the output?

Context: Example:

```
console.log([] instanceof Array); console.log([] instanceof Object);
```

Answer: `true, true` — arrays are instances of `Array`, but `Array.prototype` inherits from `Object.prototype`.

Q221: What does `new` actually do under the hood?

Context: Put these steps in order: 1. Creates a new empty object 2. Sets the object's `__proto__` to the constructor's `.prototype` 3. Calls the constructor with `this` set to the new object 4. Returns the new object (unless the constructor explicitly returns an object)

Answer: 1, 2, 3, 4.

Q222: What happens if a constructor explicitly returns an object?

Context: Example:

```
function Weird() { this.a = 1; return { b: 2 }; } const w = new Weird(); console.log(w.a); console.log(w.b);
```

Answer: `undefined, 2` — when a constructor returns an explicit object, `new` returns THAT object instead of `this`.

Q223: `Object.create` vs constructor functions: what is the output?

Context: Example: `const proto = { greet() { return "Hi " + this.name; } }; const obj = Object.create(proto); obj.name = "Venky"; console.log(obj.greet()); console.log(obj.hasOwnProperty("greet"));`

Answer: `"Hi Venky", false` — `Object.create(proto)` sets `proto` as `obj`'s prototype. `greet` is inherited (not own).

Summary Statistics

- **Total Questions:** 51
- **Categories:**
 - this Keyword: 11 questions
 - Arrow Functions: 7 questions
 - Binding: 4 questions
 - Call Stack: 5 questions
 - Heap & Memory: 6 questions
 - Execution Context: 5 questions
 - Scope & Hoisting: 6 questions
 - Prototypes & Inheritance: 6 questions